Chapter 2

Statistical Computing with R

In this chapter we introduce the computer software R that is used in this paper. The material herein will get you started with this (fantastic) statistical software, but as mentioned previously, practice is all important. If you are already familiar with R (having met it in other papers at Massey, for example) then you can skip the introductory R material if you like.

2.1 What is R?

R is a statistical software system. Rather than being a traditional statistics package, R is in fact a programming language that has many "inbuilt" statistical commands (e.g. to fit a linear regression).

R started life as a quasi-clone of the commercial package S-Plus, but development of R and S-Plus is now slowly diverging. Some of the advantages of R over other statistics packages include flexibility, power, and quality of graphical display. Another advantage is that R is free! Specifically, it is open-source software, and part of GNU project. It can be downloaded from the http://cran.r-project.org/ website. There are versions of R for all common operating systems – Windows, Linux, and MacOS. Installation is very straightforward. For Windows, for example, you download then run a Windows-style installer.

R is a New Zealand invention. It was originally developed by Ross Ihaka and Robert Gentleman in the Department of Statistics at the University of Auckland. The first (test) version of R was released in public domain in 1995. The R Development Core Team took over supervision of R in 1997. This Team includes about 20 eminent statisticians worldwide.

The remainder of this section provides a brief introduction to R. Further help with R appears in subsequent chapters. Nonetheless, if you feel that you need




Figure 2.1: Ross Ihaka (left) and Robert Gentleman.

more help getting started with this package, you might like to download a (free) R manual from http://cran.r-project.org/doc/manuals/R-intro.pdf. Alternatively, try http://cran.r-project.org/doc/contrib/Verzani-SimpleR.pdf for a (free) copy of an introductory book.

2.1.1 RStudio

Rstudio is a free and open source Integrated Development Environment (IDE) for R. It allows you to run R and execute R commands, and also provides a built-in editor to work on R scripts (see section 2.2.11). It offers many interesting advanced functionalities like version control and project management. Rstudio works on all platforms (i.e. Windows, Mac, Linux). It is a great alternative to running R from the terminal.

2.2 A First Dip into R

2.2.1 A first look at RStudio

Upon opening RStudio, you should see a window as displayed in Figure 2.2. The panel on the left is your console, i.e. where you type R commands to execute them and get the result. The top right pannel shows your environment, and will list any object that you create during your session. The bottom right panel is where you will view plots that you generate and see the help for functions, amongst other things. We will start by working on the console of RStudio, and at the end of this section, we will see how to save our commands with R scripts.

2.2.2 Setting the Working Directory

When you quit RStudio you will have the option of saving your work. If you choose to save work, R will retain:

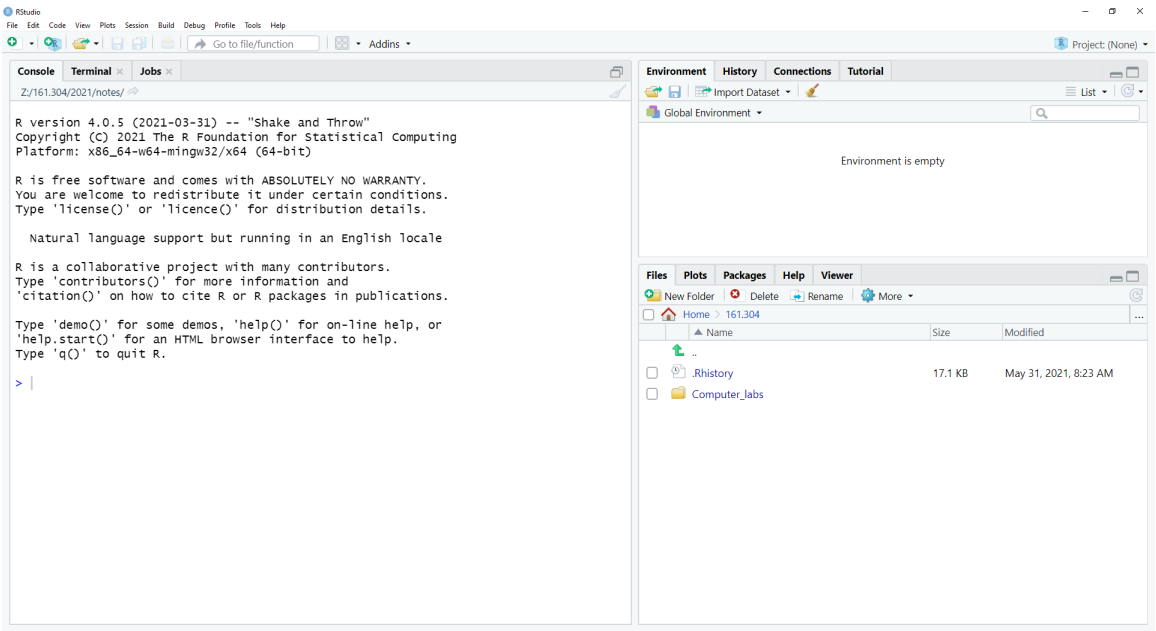


Figure 2.2: Screen shot of RStudio running under Windows 10 (should look the same with Mac or Linux).

- The *R workspace* that you have created;

- An *R history file*, listing of all the R commands that you issued.

These files are saved in your *working directory*. By default the working directory is something like `C:/Documents and Settings/mhazelto/My Documents` (i.e. your `My Documents` Folder). Sometimes it is convenient to have multiple are workspaces in a variety of folders (e.g. one for each project you work on, or one for each paper you take at Massey). In that case you will want to to save your work somewhere other than the default folder (i.e. directory).

The working directory can be changed through the `Session` menu in RStudio, by selecting `Set working directory > Choose directory`. A dialogue box will appear inviting you to enter the directory in which you wish to work. You can search through your directory tree to find your preferred folder. See Figure 2.3.

Once you have selected the working directory, R will save the workspace and history files in that directory when you quit (assuming you respond 'yes' to the save option). This will create an icon in that working directory for your R workspace. The computer should recognize R as the program associated with an R workspace. Hence double clicking an R workspace will start up R with that workspace loaded, and with the working directory set appropriately.

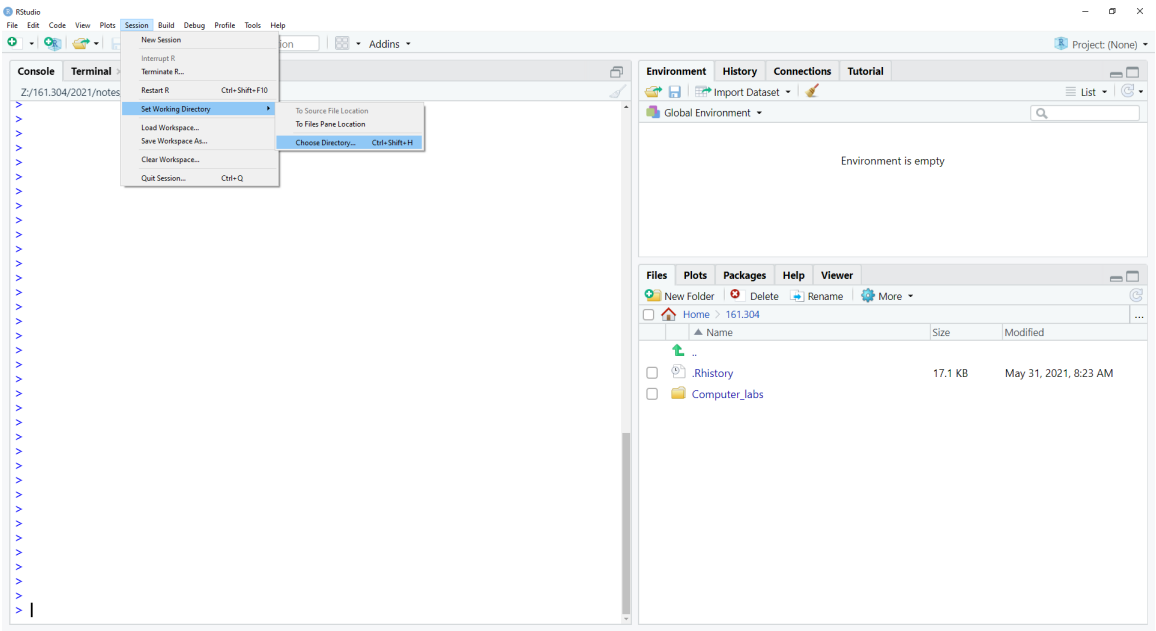


Figure 2.3: Selection of the working directory in RStudio.

2.2.3 Expressions and Assignments

Elementary commands are either *expressions* or *assignments*.

- An *expression* simply displays result of a calculation; not retained in the computer's memory.
- An *assignment* passes the result of a calculation to a variable name (or 'object') which is stored; the result is not displayed.

For example, if you type in `3+4`, and then press 'return', at the R console, you should get the following result:

```
> 3+4
[1] 7
```

In the above, the symbol `>` is the command prompt (where you type in commands). The output is on the next line. Don't worry too much at this stage about the `[1]`. Just notice that R has calculated the result of your expression as 7.

Alternatively, you can assign the result of $3 + 4$ to an R *object*, perhaps called `x`. Then just typing the `x` at the command prompt (followed by pressing the return key) will tell you the value of `x`. Viz:

```
> x <- 3 + 4
```

```
> x
[1] 7
```

Notice that in this little snippet from the R console, there was no output after the first line. The fist line is an assignment.

In the above code we used `<-` to indicate assignment from right $(3+4)$ to left (`x`). (The symbol `<-` is produced by the `<` key followed by the `-` key.) More confusingly in my opinion (but also requiring one less keystroke) you can use the `=` symbol for right to left assignment. Viz:

```
> x = 1 + 2
> x
[1] 3
```

You can also assign from left to right using `->`. (This is produced by the `-` key followed by the `>` key). For example,

```
> 3.12 -> x
> x
[1] 3.12
```

2.2.4 R Objects

All assigned variables (or any other R *objects*) are stored until overwritten or explicitly removed (deleted) by the command `rm`. To list your stored objects type `ls()` or `objects()`. Note that if using RStudio the list of stored objects is also displayed in the top right panel of the RStudio window. Consider the following snippet from an R console, for example.

```
> x <- 8
> y <- 3.1415
> ls()
[1] "x" "y"
> rm(x)
> objects()
[1] "y"
```

The first two commands create the objects `x` and `y`. The entire list of stored objects is then printed out. (I started with an empty workspace here, so only have the `x` and `y` that have just been created.) The object `x` is then removed, so that the next listing of the objects shows only `y` remaining.

You may have noticed that the commands `ls`, `objects` and `rm` are all followed by a matched pair of parentheses (in the case of `rm` containing information). All R

functions have this form of syntax. The brackets may contain information for the function (for example, `rm(x)` tells R to remove the object `x`), but the parentheses are required even when you are not supplying further information. Writing a command name without parentheses returns the R source code for the function. For example,

```
> rm(y)
> rm
function (..., list = character(0), pos = -1,.....
    inherits = FALSE)
{
    dots <- match.call(expand.dots = FALSE)...
    if (length(dots) && !all(sapply(dots, function(x) is.symbol(x) ||
        is.character(x))))
.....
```

Notice that some of the R output has been deleted to save space. This is signified by the four dots in the above R snippet.

While this use of brackets may strike you as odd at first[1] it is rather consistent. Earlier we typed `y` and found that this object contains the number 3.1415. Similarly, if we type `ls` we find that code contained in that object. The brackets are required to tell R to apply the code as an R function.

2.2.5 Vectors in R

The command `c()` (for *concatenate*) creates vectors. Consider the following R snippet, for example.

```
> x <- c(2.3,1.2,2.4)
> x
[1] 2.3 1.2 2.4
> c(x,9.0,x)
[1] 2.3 1.2 2.4 9.0 2.3 1.2 2.4
```

The first of these commands creates the vector `x` with (ordered) elements 2.3, 1.2 and 2.4. Remember that simply typing the variable name prints out its contents. We can quickly create large vectors by building on smaller ones, as you can see from the last command which concatenates `x`, 9.0, and then another copy of `x`.

It is not uncommon to want to construct a vector containing a regular sequence of numbers. While you could do this using the `c()` command there are some shortcuts as follows.

[1] Failure to remember parentheses in R code is one of the most common beginners' mistakes

- The expression `1:n` denotes the **sequence** $1, 2, \ldots, n$.
- The expression `seq(i,j,by=k)` is a sequence from i to j in steps of k.

For example:

```
> 1:5
[1] 1 2 3 4 5
> y <- seq(3,10,by=2)
> y
[1] 3 5 7 9
```

Vector arithmetic in R is conducted using the symbols `+`, `-`, `*` and `/` for addition, subtraction, multiplication and division respectively. The symbol `^` is used for exponentiation (i.e. raising to a power). One fundamental rule in vector arithmetic in R is that the package performs the aforementioned operations elementwise. Furthermore, R does not even require the vectors to be of the same length in order to perform these operations. It will recycle the elements of a short vector in order to get it up to the same length as the longer one in any computation. It is important to recognize that this is not how vectors usually behave in mathematics (where, for example, the multiplication of two vectors might be by dot or vector product). However, it turns out that elementwise operations are very convenient for performing many statistical calculations.

If that last paragraph has left you feeling a touch perplexed, some examples should help. Consider, for instance, the following R snippet.

```
> x <- c(2,3)
> y <- c(1,4,5,6)
> 2*x
[1] 4 6
> 2 + x
[1] 4 5
> y^2
[1] 1 16 25 36
> x + y
[1] 3 7 7 9
```

The calculations of `2*x` and `2+x` should be easily understood, as should `y^2` (just square the individual elements of `y`). But what about `x+y`? Well, in this case `x` is shorter than `y` so its elements must first be recycled to produce vectors of the same length. The recycled version of `x` has elements $(2, 3, 2, 3)$ (just two copies of `x` concatenated to get a vector of length 4 – the same length as `y`). We now do elementwise addition of `y` and this recycled vector. Viz:

`x` recycled	2	3	2	3
`y`	1	4	5	6
Sum	3	7	7	9

Hopefully the following snippet will now make sense.

```
> v <- c(3,5)
> w <- c(2,4,6)
> v*w
[1]  6 20 18
Warning message:
In v * w : longer object length is not a multiple of shorter object length
```

Notice that here we need only recycle the first element of `v` so that the recycled vector has elements $(3, 5, 3)$. R provides a warning message after the calculation, since the length of `w` is not an integer multiple of the length of `v`. This is allowable, but at times indicates that you are computing something silly. I have certainly found that warning useful when I have made mistakes in R!

So far we have focused entirely on numeric vectors. However, R also understands vectors of:

- characters
- logical values `TRUE` and `FALSE`; abbreviations `T`, `F` allowed.
- factors (i.e. categorical variables)

This is perhaps must easily illustrated through some examples.

```
> mywords <- c("This","is","a","character")
> mywords
[1] "This"      "is"        "a"         "character"
> c(F,T,F,F)
[1] FALSE TRUE FALSE FALSE
> factor(c("Low","Low","Medium","High","High"))
[1] Low    Low    Medium High   High
Levels: High Low Medium
```

The most important thing to recognize here is the difference between a character vector and a factor (categorical variable). A character vector has words as elements. If you change the word, you change the information the vector contains. So, for example, the character vectors `c("apples")` and `c("oranges")` are naming different fruit. However, in a factor the words are merely labels, and you don't really lose information by changing the labels. So for instance I could have replaced that factor vector from the above R snippet with

```
factor(c("L","L","M","H","H"))
```

or

```
factor(c(1,1,2,3,3))
```

and I would still convey essentially the same information so long as you remembered the 'coding' (or labelling) that you had used for the different categories.

Numerical vectors can be compared by inequalities (with `==` denoting equality) to produce logical vectors. You can also use `!=` for not equal, `&` for logical AND, and `|` for logical OR. Comparisons are done elementwise using these operations. For example:

```
> (1:5) == (5:1)
[1] FALSE FALSE  TRUE FALSE FALSE
> (1:5) > (5:1)
[1] FALSE FALSE FALSE  TRUE  TRUE
> ( (1:5)==(5:1) ) | ( (1:5)>(5:1) )
[1] FALSE FALSE  TRUE  TRUE  TRUE
> ( (1:5)==(5:1) ) & ( (1:5)>(5:1) )
[1] FALSE FALSE FALSE FALSE FALSE
```

2.2.6 Indexing Vectors

You will often want to extract one or more elements from a vector. To *index* components of a vector `x`, use the form `x[...]`. The square brackets can contain:

- numeric vector specifying elements;
- logical vector: only `TRUE` elements required.

You may also be interested in finding which elements of a vector satisfy a certain property. This can be done using the function `which`.

Consider the following snippet.

```
> x <- c(1.1,3.2,4.3,7.4)
> x[c(2,4)]
[1] 3.2 7.4
> x[-2]
[1] 1.1 4.3 7.4
> x[x > 3.5]
[1] 4.3 7.4
> which(x > 3.5)
[1] 3 4
```

The syntax `x[c(2,4)]` returns a vector containing just the second and fourth elements of `x`. The syntax `x[-2]` returns a vector with all elements except the second. You could use `x[-c(1,3)]` to return all elements of `x` except the first and third. The syntax `x[x > 3.5]` returns all elements of `x` which are greater than 3.5, while `which(x > 3.5)` tells you than only the third and fourth elements of `x` are greater than 3.5.

2.2.7 Data frames

A *data frame* is a collection of (column) vectors each of the same length. The vectors may be numeric, factor, or whatever. Each particular column and row of a data frame is given a name which can be chosen by the user, or assigned a default by R.

Data frames are a natural way of storing information on multiple variables for multiple individuals. Each column vector contains information on one variable (e.g. age, sex). Each row of the data frame corresponds to a single individual.

Consider the following R snippet.

```
> employee <- c("Dilbert", "Wally", "Catbert", "TheBoss")
> job <- factor(c("Engineer","Engineer","Manager", "Manager"))
> x <- c(8,1,NA,-2)
> dilbert <- data.frame(employee, job, competence=x)
> dilbert
  employee       job  competence
1  Dilbert  Engineer           8
2    Wally  Engineer           1
3  Catbert   Manager          NA
4  TheBoss   Manager          -2
```

Here we have created a data frame on four individuals (from the excellent cartoon strip 'Dilbert' – see http://www.unitedmedia.com/comics/dilbert/). Each row of the data frame relates to one individual, each column to one variable. The names of these individuals are listed under the variable `employee`. Their jobs are listed as a factor (remember, that's a categorical variable) and their competence in the workplace is given as a score out of ten. The `NA` symbol specifies a missing value. (I can't make up my mind about the competence of Catbert.) Notice that the columns (variables) inherit the original vector name unless you explicitly rename them (as is done in the case of the vector `x`, renamed `competence`).

The variables (columns) of a data frame cannot be accessed by name until the data frame has been *attached* with `attach`. Think of a data frame like a glass containing the variables. Before the data frame has been attached, the glass is wrapped in a brown paper bag. R cannot see inside it, and so cannot extract information on the variables in the data frame (unless you tell ask properly! See below). Once the data frame is attached the glass is taken out of the bag. R now can see inside the glass and so can return information on the variables in the data frame.

As an example, consider the following snippet.

```
> rm(employee,job,x)
> dilbert$competence
```

```
[1]  8  1  NA  -2
> job
Error: Object "job" not found
> attach(dilbert)
> job
[1] Engineer  Engineer  Manager  Manager
Levels:  Engineer  Manager
> detach(dilbert)
```

The first command removes the original variables. The syntax `A$B` indicates variable `B` in data frame `A`. This is what I meant when I said you have to ask R properly for a variable if you haven't attached the corresponding data frame. R can find `competence` when you explicitly tell it to look in the data frame `dilbert`; see the second line of code. However, if you just ask for `job` without any indication of where R should look, it cannot find this variable; see the third command in the snippet. After the data frame `dilbert` is attached, R can find `job` by the name of the variable alone. Finally, note that `detach` has the reverse effect of `attach`, hiding the variables within the data frame `dilbert`.

2.2.8 Importing Data

Suppose you have some data that you wish to analyse in R. If the data set is very small then you could type it in by hand, using the `c` function to create the necessary variables (vectors). However, if you already have the data stored electronically in some format then an automatic method of reading the data in R will be much more convenient (and much less prone to transcription errors). While R has functions to read in data from a wide variety of file types (such as Excel spreadsheets), we will focus here on importing data from plain text files. This should be sufficient for almost all intents and purposes, since every sensible spreadsheet package will have an option to save data as a plain text file.

The `scan` command reads in text from a file as a single variable. The `read.table` command is more flexible, importing data in tabular form and storing the result as a data frame. Consider for example the following snippet.

```
> ibm <- scan(file="ibm.txt")
Read 250 items
> ibm
  [1] 64.37 62.50 63.50 63.37 63.12 67.37 65.37 .....
.....
[239] 68.12 66.50 67.87 68.00 68.37 67.50 66.37 .....
> life <- read.table(file="life.txt", header=T)
> life
        Country LifeExp People.per.TV People.per.Dr  .....
1     Argentina    70.5           4.0           370  .....
```

```
.....
40        Zaire    54.0           NA         23193  .....
```

The file `ibm.txt` is stored in the same folder as the R workspace (otherwise it would be necessary to specify the file name by its full path, e.g. `C:/Documents and Settings/user/ibm.txt`). This text file contains the price of IBM stock on 250 consecutive trading days in 1980. The first command scans these data into R and stores the resultant vector as the object `ibm`. (The name was chosen for conveniences – it does not have to match the text file name.)

The file `life.txt` contains data on life expectancy (and potentially related socio-economic variables) for 40 countries, stored in tabular format. These are read in by the `read.table` command and stored as a data frame called `life`. The `header=T` argument to the command tells R that the first row of the text file contains variable names rather than data themselves.

Both the text files `ibm.txt` and `life.txt` are available on the paper Stream site for download. You might like to download them onto your home computer and practice reading the data into R as described above.

2.2.9 Editing Data

Suppose that you discover that your data set contains one or more errors. You can edit values in a number of ways. For example, you can simply reassign the erroneous elements of a vector. Consider the following snippet.

```
> ibm[5]
[1] 63.12
> ibm[5] <- 65.12
```

Here the 5th value of the IBM data set is incorrect. The second command assigns the correct value of 65.12, which overwrites the original incorrect value (of 63.12).

The square bracket method of indexing vectors can be extended to index elements of a data frame (or matrix). Specifically, `A[i,j]` is the i, jth element (row i, column j) of the data frame `A`. The syntax `A[i,]` returns the entire ith row, while `A[,j]` returns the jth column. Consider the following snippet:

```
> life[2,]
     Country LifeExp People.per.TV People.per.Dr .....
2 Bangladesh    53.5           315          6166 .....
> life[2,4]
[1] 6166
> life[2,4] <- 7166
```

Here we have corrected the number of people per doctor in Bangladesh from 6166 to 7166.

As an alternative, R has a rudimentary spreadsheet capability through the `edit` command. For example, typing

```
life <- edit(life)
```

produces a simple spreadsheet representation of the data frame life which can be edited in the obvious way. See Figure 2.4. When you have finished with the spreadsheet just click on the cross in the top right hand corner to close, and the resulting changes will be assigned to `life`.

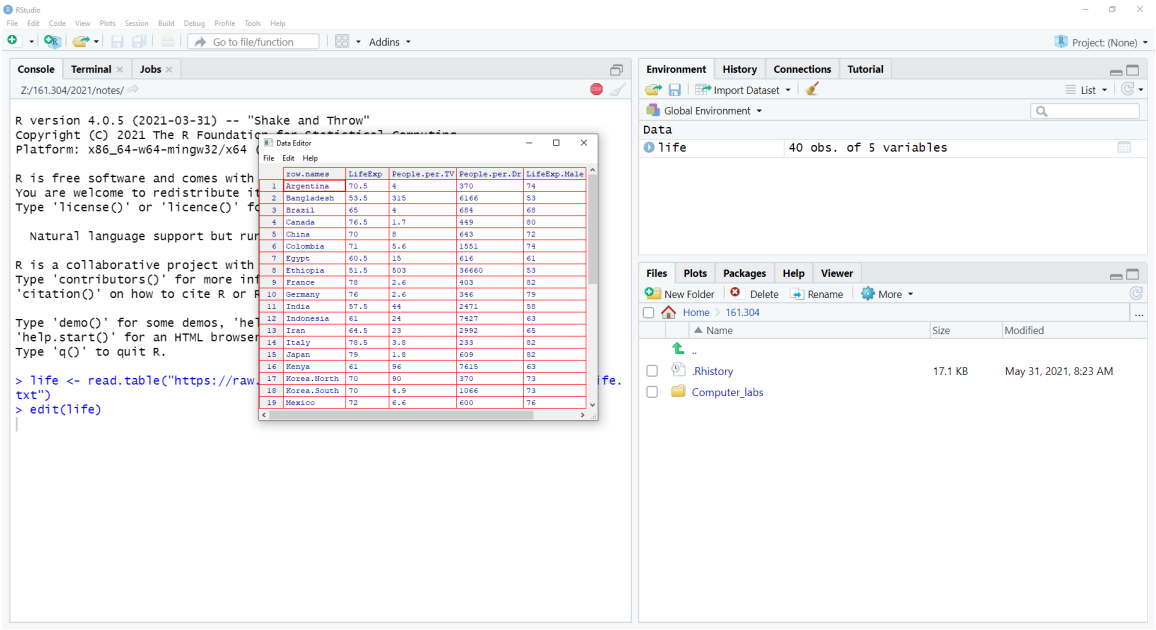


Figure 2.4: Spreadsheet for editing R data frames.

2.2.10 R Packages

R objects (functions, data etc.) are organized in libraries/packages. When you first install R it comes with a variety of packages. Some of these are loaded by default when R starts each time. This is important, because you only have access to functions in loaded packages. For example, the function `ls` that you met earlier is part of the `base` package that is automatically loaded, and so you may use that function immediately. On the other hand, the function `mvrnorm` (which produces simulations from multivariate normal distributions) is part of the `MASS` package which is not loaded by default. If you try to use this function immediately after starting R, you will get the following:

```
> mvrnorm(1,mu=0,Sigma=1)
Error: object 'mvrnorm' not found
```

To access an object like `mvrnorm` which is not in a default package, you must first load the requisite package. This is done using the `library` command. For example, the code

```
> library(MASS)
> mvrnorm(1,mu=0,Sigma=1)
[1] 0.7866292
```

first loads the package `MASS`, and the calls the function `mvrnorm`. Note that once loaded, a package remains so for the rest of the R session. However, once you quit R and then restart, you will need once again to load the package `MASS` in order to use the `mvrnorm` function.

One of the strengths of R is that it has a huge number of packages, covering a wide variety of statistical methods and tools. Most of these packages are not part of the initial R installation. The package `coda` (which handles out from Markov chain Monte Carlo methods) is a case in point. Try typing `library(coda)` and you will get an error telling you that there is no package of that name (unless you installed it earlier).

In such cases you need to install the package into your version of R. This is the process of loading the package onto your computer system, for where it may then be loaded into R using the `library` command as discussed above. In principle the easiest way to install a new package is to go to the `Packages` window in the bottom-right panel of RStudio, click on the `Install` button in the top left corner of that window, and type in the name of the package that you want to install (See Figure 2.5). By default, RStudio will search for the package in the CRAN repository, which is the main (but not the only one) repository for R packages. Occasionally, you may be asked to select a mirror site; in such case, select a local mirror, say in Australia or New Zealand. Package installation will then proceed automatically.

A problem will occur with the above procedure if your internet connection uses a proxy server. In that case, one solution is to go to the packages webpage `http://cran.r-project.org/web/packages/`, select the page with the package that you want, and download the package as a `.tar.gz` archive. Save the `.tar.gz` file somewhere on your system, then select `Install from Package Archive File...` from the aforementioned `Install Packages` menu. See the R FAQ page `http://cran.r-project.org/bin/windows/rw-FAQ.html` for an alternative method of dealing with proxy server problems.

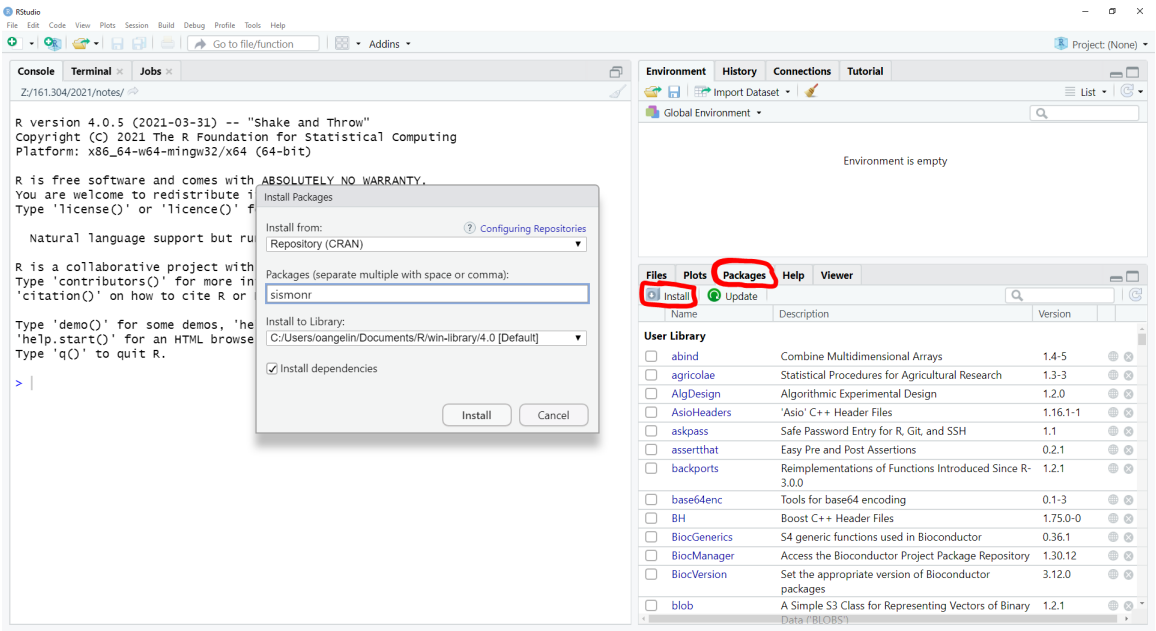


Figure 2.5: Screen shot of RStudio when installing the `sismonr` package.

2.2.11 Managing Your Code

You will make mistakes when you code in R. Everyone does. Do not be disheartened – things really will improve with practice!

One way of speeding up your coding with R is to save a listing of (successful) commands in a file. These may be reused later (if you want to replicate an analysis), or modified for new purposes. R provides 'script' files for this purpose. You can open a new or existing script from RStudio's `File` menu, or by clicking on the page with a green cross icon on the far left of the icon bar, and select `R script`. An R script is simply a text file that can be used to record code, but has the benefit that code from that file can be run directly by simply highlighting the required block of code, or clicking anywhere on the line you want to execute, and then hitting `CTRL-Enter`, or pressing the `Run` button on the top left of the editor panel, See Figure 2.6. You can include comments in a script file that will not be executed by prefacing them with a `#` symbol.

2.2.12 Some R Functions to Get You Started

Here are a few basic commands to get you started with R.

- `help` accesses R's help system; e.g. `help(ls)`.

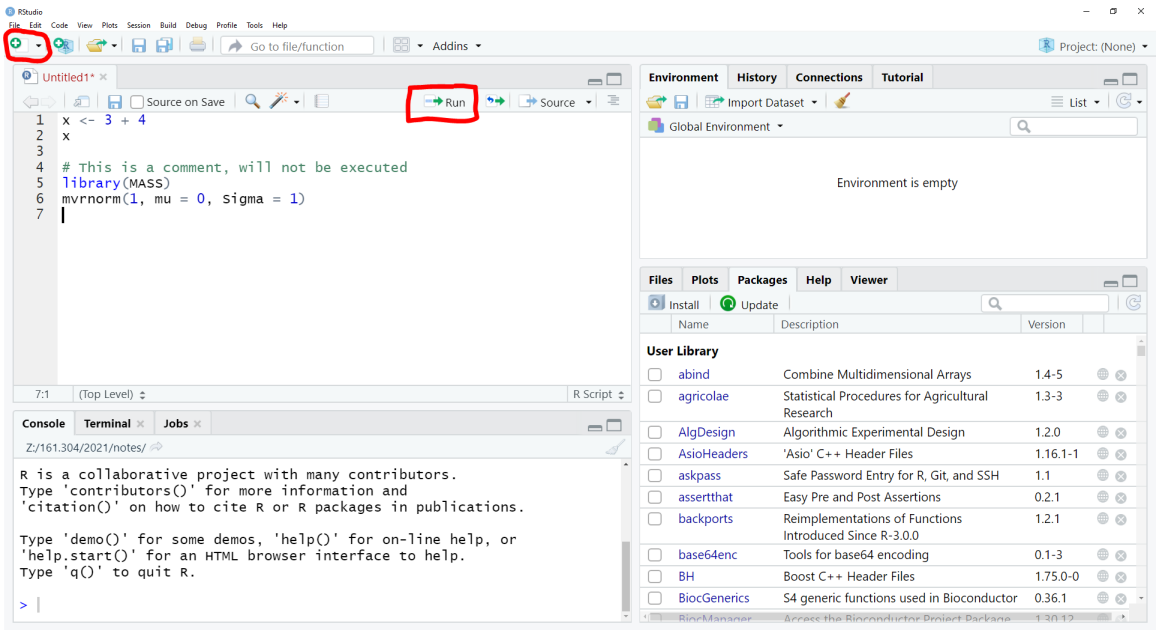


Figure 2.6: Screen shot of RStudio with script file. The button to create a new R script and the button to execute a command from the R script are circled in red.

- `mean`, `sd`, `min`, `max` and `range` give mean, standard deviation, minimum, maximum and range respectively for a vector argument.
- `var` returns variance of a vector argument, or the covariance (dispersion) matrix for a matrix argument.
- `summary` returns summary information dependent on argument type.
- `plot` produces a plot on the current graphics tool. The type of plot depends on the type of argument. Simplest is `plot(x,y)` which produces a scatterplot of vectors `x` and `y`.

Practice is the key when learning R. Play around with the package and get used to the syntax. At appropriate time throughout the chapter, you will be prompted to work through R Computer Lab sheets. These Exercise Sheets are an absolutely essential element of the course, and should not be skipped!

2.3 Practice Exercises

Exercise 1

1. Write (efficient) code to create the following sequences:

(a) $2, 4, 6, \ldots, 100$ (even numbers up to 100)

(b) $1, 2, 3, 4, 5, 4, 3, 2, 1$

(c) $10, 20, 30, 40, 10, 20, 30, 40, 10, 20, 30, 40$
(hint: `rep(x,n)` is equivalent to `c(x,x,.....,x)`, where `x` is repeated `n` times).

2. What would be the result of the following calculations using R?

(a) `c(4,2,4)^2 - c(1,3)`

(b) `(1:5)^(1:2)`

Exercise 2
A sequence $y_1, y_2, \ldots, y_n$ of $n = 100$ standard normal random variables can be *simulated* by `y <- rnorm(100)`.

Write code to return:

1. $y_2, y_4, y_6, \ldots, y_{100}$
2. all elements of y greater than 1.96
3. the indices of all negative elements of y
4. $\frac{y_{51}}{y_1}, \frac{y_{52}}{y_2}, \ldots, \frac{y_{100}}{y_{50}}$

Solutions to these Exercises appear in section 2.4.

You should now do Computer Lab 1.

2.4 Solutions to Practice Exercises

This section contains solutions to the exercises in Section 2.3. The solutions to problems requiring R code are by no means unique. There are alternatives, some of which will be as efficient as the suggestions below.

Exercise 1

1. (a) `(1:50)*2` or `seq(2,100,by=2)`.
   (b) `c(1:5,4:1)`.
   (c) `rep(1:4,3)*10`.
2. (a) $(15, 1, 15)$.
   (b) $(1, 4, 3, 16, 5)$.

Exercise 2

1. `y[seq(2,100,by=2)]`
2. `y[y > 1.96]`
3. `which(y < 0)`
4. `y[51:100]/y[1:50]`